

# **NeuroClip – Context Aware Video Processing**



**Ali Javed (FC-200)**

**Asad Sardar (FC-211)**

*Supervised By*

***Subhan Arif***

*Submitted for the partial fulfillment of BS Computer Science degree to  
the Faculty of Engineering & Computing.*

DEPARTMENT OF COMPUTER SCIENCE

**NATIONAL UNIVERSITY OF MODERN LANGUAGES**

**FAISALABAD**

**December, 2025**

## ABSTRACT

The efficient retrieval of certain information on the long-form video material is a vital challenge within the rapidly developing framework of digital media. The workflows existing in use are not optimized in computing power and often an identical search has to be reprocessed each time the search occurs. NeuroClip can solve these fundamental deficiencies through the provision of an intelligent, intent aware video processing platform which has the capability of deriving knowledge that are searchable as transcripts. The project is a major designation of operational efficiency in the fields of education through the conversion of traditional video archives into dynamic and queryable display of video information resource and while also focusing on semantic context rather than on mere keyword hits. The approach being employed is transcript-based methodology where audio is transcribed, then the information is turned into a form of a vector that is useful in accessing the context and then this information is subjected to a windowed search mechanism with optional cross-encoder re-ranking to construct coherent clips. The technical implementation is based on a stack that is modern: a responsive web interface that is written using React and a scalable backend using FastAPI. Core intelligence makes use of sentence-transformers (e.g. MiniLM-L6-v2) and pgvector implemented in SupaBase to efficiently store vectors, and ffmpeg+ to daily conduct accurate clip extraction and streaming. System test entailed both black-box parameter sweeps and grounded relevance tests to test the retrieval accuracy under different window sizes as well as window stride configurations. Latency Coherence Evaluations were done on a standard workstation (4+ core processor, 16GB RAM). The experimental findings have proven that with NeuroClip, history queries can be answered in under a second and clips related to contextual level can be retrieved in several seconds. Although effective, the existing system has its shortcomings, especially its relying on the quality of Automated Speech Recognition (ASR) the transcript of which can lead to the deterioration of search accuracy. To improve on these problems, future directions include supporting visual cues with multi-modal embeddings, supporting query rewrites with Large Language Models (LLMs) to streamline and support real-time generation of clips of video.

# CERTIFICATE

Dated: \_\_\_\_\_

## Final Approval

It is certified that the project report titled '**NeuroClip – Context Aware Video Processing**' submitted by **Ali Javed and Asad Sardar** for the partial fulfillment of the requirement of "**Bachelor's Degree in Computer Science**" is approved.

## COMMITTEE

Dr. Noman Malik

Dean CS

Signature: \_\_\_\_\_

Dr. Pervez Akhtar

HoD CS

Signature: \_\_\_\_\_

Kalsoom Fatima

Head Project Committee

Signature: \_\_\_\_\_

Subhan Arif

Supervisor

Signature: \_\_\_\_\_

## DECLARATION

We hereby declare that our dissertation is entirely our work and genuine / original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

### Group Members

### Signature

1. Ali Javed

---

2. Asad Sardar

---

## PLAIGRISM CERTIFICATE

This is to certify that the project entitled “**NeuroClip – Context Aware Video Processing**” is being submitted here for the award of the “**Degree of Bachelor of Science**” in “**Computer Science**”. This is the result of the original work by **Ali Javed and Asad Sardar** under my supervision and guidance. The work embodied in this project has not been submitted earlier for the award of any degree or any other diploma, certificate, or similar title to the best of my knowledge and belief.

### Turnitin Originality Report

Processed on: Dec 7, 2025, 9:43 PM GMT+5

ID: 2838528273

Word Count: 9478

Similarity Index

10%

Similarity by Source

Internet Sources: 08%

Publications: 02%

Student Papers: 09%

Date: 08/12/2025

---

Subhan Arif

# TURNITIN ORIGINALITY REPORT

**“NeuroClip – Context Aware Video Processing”** by **Ali Javed** and **Asad Sardar** under the supervision of **Subhan Arif**

Processed on: Dec 7, 2025, 9:43 PM GMT+5

ID: 2838528273

Word Count: 9478

Similarity

Index

10%

Similarity by Source

Internet Sources: 08%

Publications: 02%

Student Papers: 09%

## \*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

### Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

### Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

## Frequently Asked Questions

### How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (\*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

### What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



## FYP

### ORIGINALITY REPORT

|                  |                  |              |                |
|------------------|------------------|--------------|----------------|
| <b>10%</b>       | <b>8%</b>        | <b>2%</b>    | <b>9%</b>      |
| SIMILARITY INDEX | INTERNET SOURCES | PUBLICATIONS | STUDENT PAPERS |

### PRIMARY SOURCES

|          |                                                                                                                                                                                                     |               |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <b>1</b> | <b>Submitted to Higher Education Commission Pakistan</b><br>Student Paper                                                                                                                           | <b>5%</b>     |
| <b>2</b> | <b>Submitted to Strathmore University (Main Account)</b><br>Student Paper                                                                                                                           | <b>1%</b>     |
| <b>3</b> | <b>numlrwp.numl.edu.pk</b><br>Internet Source                                                                                                                                                       | <b>&lt;1%</b> |
| <b>4</b> | <b>Submitted to Misr International University</b><br>Student Paper                                                                                                                                  | <b>&lt;1%</b> |
| <b>5</b> | <b>sda.gov.sa</b><br>Internet Source                                                                                                                                                                | <b>&lt;1%</b> |
| <b>6</b> | <b>GuangLi Wu, Shanshan Song, Xingyue Wang, Jing Zhang. "Reconstructive network under contrastive graph rewards for video summarization", Expert Systems with Applications, 2024</b><br>Publication | <b>&lt;1%</b> |
| <b>7</b> | <b>ajith.softcomputing.net</b><br>Internet Source                                                                                                                                                   | <b>&lt;1%</b> |
| <b>8</b> | <b>www.theseus.fi</b><br>Internet Source                                                                                                                                                            | <b>&lt;1%</b> |
| <b>9</b> | <b>Jeshmitha Gunuganti, Zhi-Ting Yeh, Jenq-Haur Wang, Mehdi Norouzi. "Unsupervised video summarization with adversarial graph-based attention network", Journal of Visual</b>                       | <b>&lt;1%</b> |



## TABLE OF CONTENTS

|                                                   |    |
|---------------------------------------------------|----|
| CHAPTER 1 .....                                   | 1  |
| 1.1 Introduction .....                            | 2  |
| 1.2 Project Domain.....                           | 2  |
| 1.3 Problem Identification .....                  | 3  |
| 1.3.1 Proposed Solution .....                     | 3  |
| 1.3.2 Objectives .....                            | 4  |
| 1.3.3 Scope of the Project .....                  | 4  |
| 1.4 Effectiveness of the System .....             | 5  |
| 1.5 Resource Requirement .....                    | 5  |
| 1.5.1 Hardware Requirement .....                  | 5  |
| 1.5.2 Software Requirement .....                  | 6  |
| 1.5.3 Data Requirement .....                      | 7  |
| 1.6 Report Organization .....                     | 7  |
| CHAPTER 2 .....                                   | 9  |
| 2.1 Related Literature Review.....                | 10 |
| 2.2 Related Systems .....                         | 11 |
| 2.3 Identified Problem from Existing Work.....    | 13 |
| 2.4 Selected Boundary for Proposed Solution ..... | 13 |
| CHAPTER 3 .....                                   | 14 |
| 3.1 System Specification .....                    | 15 |
| 3.2 System Modules .....                          | 15 |
| 3.2.1 User Registration & Login.....              | 15 |
| 3.2.2 Video Upload & Management.....              | 16 |
| 3.2.3 Video Summarization .....                   | 16 |
| 3.2.4 Video Summarization in Text.....            | 16 |

|                                                         |    |
|---------------------------------------------------------|----|
| 3.2.5 Sensitive Information Blurring .....              | 16 |
| 3.2.6 Video Compression.....                            | 17 |
| 3.3 Functional Requirements.....                        | 17 |
| 3.3.1 User Registration and Authentication .....        | 17 |
| 3.3.2 Video Upload and Preprocessing.....               | 17 |
| 3.3.3 Automated Video Summarization.....                | 18 |
| 3.3.4 Semantic Search and Retrieval .....               | 18 |
| 3.3.5 Sensitive Information Redaction.....              | 18 |
| 3.3.6 Video Compression and Playback .....              | 18 |
| 3.4 Non-Functional Requirements .....                   | 19 |
| 3.4.1 Usability .....                                   | 19 |
| 3.4.2 Reliability & Availability .....                  | 19 |
| 3.4.3 Performance & Reactiveness .....                  | 19 |
| 3.4.4 Security & Privacy .....                          | 20 |
| 3.4.5 Portability & Compatibility .....                 | 20 |
| CHAPTER 4 .....                                         | 21 |
| 4.1 System Design and Analysis .....                    | 22 |
| 4.2 Use Case Diagrams .....                             | 25 |
| 4.3 Detailed Use Case .....                             | 25 |
| 4.3.1 Search and Clip Merge - Full Dress Use Case ..... | 26 |
| 4.4 Activity diagram:.....                              | 28 |
| 4.5 Data Flow Diagram: .....                            | 31 |
| 4.5.1 DFD Level 0 (Context Diagram) .....               | 31 |
| 4.5.2 DFD Level 1 .....                                 | 31 |
| 4.6 System Sequence Diagram.....                        | 32 |
| 4.7 Sequence Diagram.....                               | 33 |
| 4.8 Design Class Diagram .....                          | 34 |

|                  |                              |    |
|------------------|------------------------------|----|
| 4.9              | Architectural diagrams ..... | 35 |
| 4.9.1            | Interface Design .....       | 36 |
| 4.9.2            | Component Level design ..... | 37 |
| REFERENCES ..... |                              | 39 |

## LIST OF FIGURES

|                                            |    |
|--------------------------------------------|----|
| Figure 4.1: Main Use Case Diagram.....     | 25 |
| Figure 4.2: Activity Diagram.....          | 30 |
| Figure 4.3: DFD Level 0.....               | 31 |
| Figure 4.4: DFD Level 1.....               | 32 |
| Figure 4.5: System Sequence Diagram.....   | 33 |
| Figure 4.6: Sequence Diagram.....          | 34 |
| Figure 4.7: Class Diagram .....            | 35 |
| Figure 4.8: Interface Design Diagram ..... | 37 |
| Figure 4.9: Component Level Design.....    | 38 |

## **LIST OF TABLES**

|                                                 |    |
|-------------------------------------------------|----|
| Table 1.1: Hardware Requirement .....           | 6  |
| Table 1.2: Software Requirements .....          | 7  |
| Table 2.1: Summary of Reviewed Literature ..... | 11 |
| Table 2.2: Summary of Existing System .....     | 12 |
| Table 4.1: Full Dress Use Case .....            | 26 |

# **CHAPTER 1**

## **INTRODUCTION**

## 1.1 Introduction

Video is the media that is the most successful in the fast changing digital media landscape, as the means through which information spreads, people are entertained and even watched over. The explosion of video material on learning platforms, company repositories and social networks has brought about a tremendous necessity of effective retrieval systems which extend beyond basic system of metadata filtering. Although this may cause a rise, it still does not mean that it is easy to access certain information within long-form video content. Conventional video search systems largely depend on keyword matching which is by its nature fragile, and fails to meet the semantic context of a query by the user. This usually ends up retrieving piece meal or irrelevant fragments which do not effectively answer the purpose of the user. Moreover, the traditional work process entails manual sifting through volumes of footage and this process is tedious and can be subject to errors. The current solutions are also ineffective in the use of computer time as it is frequently necessary to reform videos and reread them every time new search is needed hence wasting precious computer time. It is these limitations that indicate that there is now an immediate requirement to have an intelligent, intent-aware system capable of extracting searchable knowledge within video content, but that puts emphasis in context and topic coverage, as opposed to keyword hits.

NeuroClip fills these very important gaps by adding a powerful, end-to-end video processing solution which can convert stagnant video collections into interactively queryable assets. The system incorporates high-quality pipeline of video ingestion, automated transcription and sentence level embedding computation to enable semantic deep comprehension. The windowed retrieval process with optional cross-encoder reranking, in contrast to existing keyword-based tools, which tend to deliver very fragmented retrievals, is that NeuroClip uses a windowed retrieval process into coherent, context-dependent clips. This system has the benefit that it uses a common job identifier to unify video metadata, embeddings and history, which enables computationally expensive artifacts to be stored and reused. This is markedly better than conventional workflows as it relies on no redundancy in processing which increases the reproducibility and efficiency of work among the users in academic, business and surveillance fields.

## 1.2 Project Domain

NeuroClip works at the edge of Artificial Intelligence (AI), Deep Learning and Computer Vision in particular in the smart summaries and retrieval of video content. The project

solves the important problem of traveling through long-form video archives by combining Natural Language Processing (NLP) and machine-learned embeddings so as to facilitate semantic comprehension of user queries. By automating the transcription process and using vector-based similarity search, in turn, the system addresses the issue of inefficient manual scrubbing and search limitations with keywords. This solution helps to bridge the gap between the static storage of media and the dynamic information access allowing the users extract the contextually relevant segments and coherent summaries from a vast amount of video footage, thereby improving the field of intelligent media processing and automated content management

### **1.3 Problem Identification**

Current video retrieval systems primarily utilize metadata tags or basic keyword matching to locate content within large archives, with some platforms offering limited transcript-based search capabilities. However, these traditional approaches are inherently brittle; they fail to grasp the user's actual intent or the semantic context behind a query, often resulting in the retrieval of disjointed, short, or irrelevant video fragments that do not fully address the user's needs. Furthermore, users are typically forced to manually scrub through hours of footage to locate specific information, which is a labor-intensive and error-prone process that significantly hampers productivity. From a computational standpoint, existing workflows often suffer from severe inefficiency, requiring videos to be reprocessed or re-analyzed for every new search instance, leading to redundant resource usage. This project aims to address these limitations by developing a context-aware system that understands semantic meaning, assembles coherent video clips rather than fragmented hits, and optimizes computational resources through the reuse of stored embeddings.

#### **1.3.1 Proposed Solution**

The proposed solution, NeuroClip, is an encompassing video processing platform aimed to turn static video archives into dynamic and searchable knowledge bases through the use of advanced techniques of vector embedding. The whole process of video ingestion, transcription, and vectorization is automated, and is transformed into sentence-level embeddings that capture the meaning of the video, not just the keywords. Unlike a traditional search tool, NeuroClip leverages a complex windowed retrieval mechanism with cross-encoder reranking for identifying, ranking, and merging the relevant video segments into coherent and playable video clips that reflect the intent of the user accurately. To solve computational inefficiencies, the solution applies a strong storage



architecture identified by unique job identifiers to store and reuse the expensive embeddings for future queries. This approach not only dramatically reduces processing time but also a scalable and responsive environment for the user to explore and retrieve specific information from a long form content.

### **1.3.2 Objectives**

The key objectives of NeuroClip are:

- To design a system that delivers intent-aware clip retrieval by prioritizing semantic context and topic coverage over traditional brittle keyword matching.
- To develop a comprehensive video processing platform that integrates automated transcription, sentence-level embedding, and configurable parameters for precise clip generation.
- To improve the efficiency of video analysis workflows by enabling the caching and reuse of stored embeddings, thereby eliminating the need for redundant reprocessing.
- To reduce the time and effort required to locate specific information within long-form video content through a windowed semantic search mechanism.
- To increase the reproducibility of search results by implementing a robust history management system keyed by stable job identifiers.

To validate the developed system through rigorous parameter sweep testing and grounded relevance assessments to ensure high retrieval accuracy and sub-second latency.

### **1.3.3 Scope of the Project**

The scope of NeuroClip includes the creation of a full-stack web application that will help to facilitate the intelligent retrieval and management of video contents. The boundary of this system is marked by the integration of its core pipeline combining video ingestion, automated speech to text transcription and generation of sentence-level embeddings for deep semantic search capability. Key functionalities provided by the system are the ability to upload different formats of videos, the ability to perform context-aware queries using natural language, and the ability to automatically generate and stream coherent video clips based on user intent. The system is intended to function in the web-based environment, making use of a reactive React frontend for interacting with the system and a FastAPI backend for processing, keeping accessibility available in common

web browsers. Furthermore, it is included in the scope to implement a persistent storage layer using the features of Supabase and pgvector for the management of the metadata of the videos and the embedding of vectors used to keep the history and to search it in a reproducible way, while explicitly excluding features that are not in the scope, such as the training of a proprietary ASR model, or the use of some complex non-linear video editing workflow.

## **1.4 Effectiveness of the System**

Unlike traditional video search functionalities that rely on exact match with keywords, which may lead to fragmented and irrelevant results, NeuroClip brings a new paradigm by using the concept of deep semantic understanding for understanding the intention of the user. This contribution in effect closes the gap between static storage of content and dynamic retrieval of information, which has a clear advantage over manual scrubbing methods that are time consuming and error-prone. The main usefulness of the system is in the ability to dramatically reduce the time and cognitive effort needed to extract specific information from long form video archives. NeuroClip is a priceless resource to any sort of application, whether it be in educational research where students need to extract and assemble specific lecture content, corporate integration where efficient meeting transcription is required, or the media sector where fast content extraction is necessary. Furthermore, the architecture of the system guarantees operational efficiency by reusing the embeddings that are stored, making it a very efficient and scalable system for managing large digital video libraries.

## **1.5 Resource Requirement**

This section describes the basic technical resources required for the successful design, training, development and deployment of the NeuroClip system. It classifies the specific hardware configurations and software tools along with the data assets needed to construct and use the computationally intensive AI-enabled video processing architecture.

### **1.5.1 Hardware Requirement**

In order to develop and deploy the NeuroClip system, a powerful hardware setup is required, in order to be able to cope with the computational load of video processing, transcription, and vector embedding generation. A minimum of a 4-core work station processor is essential as the application requires the abilities to handle the concurrent processes of the FastAPI backend and React frontend. Sufficient Random Access

Memory (RAM) is important to load the deep learning models (in this case sentence-transformers) as well as large video files in memory without performance bottlenecks. In addition, sufficient disk space is needed to store the project repository, uploads of the videos, and the postgres database using the pgvector extension. While the system is designed to be optimized for being run on the CPU, it is recommended to use a dedicated GPU to speed up inference times in the phase of computing the embeddings. The detailed hardware specifications that have been utilized for this project are given in Table 1.1.

Table 1.1: Hardware Requirement

| Component              | Minimum Requirement                      | Recommended Specification                  |
|------------------------|------------------------------------------|--------------------------------------------|
| <b>Processor (CPU)</b> | Quad-core (e.g., Intel i5 or equivalent) | Hexa-core or higher (Intel i7/Ryzen 5)     |
| <b>Memory (RAM)</b>    | 8 GB                                     | 16 GB or higher                            |
| <b>Storage (Disk)</b>  | 50 GB Free Space (SSD recommended)       | 256 GB NVMe SSD                            |
| <b>Graphics (GPU)</b>  | Integrated Graphics                      | Dedicated GPU (NVIDIA GTX/RTX series)      |
| <b>Network</b>         | Broadband Internet Connection            | High-speed Internet for API/Cloud DB calls |

### 1.5.2 Software Requirement

The development of NeuroClip is based on a modern fast software environment that is built on a full stack architecture scalable to high performance. The frontend interface is developed with the help of React Library, which is leveraged by Vite, a fast build tooling framework, and TypeScript, a Type Safety features, while TailwindCSS is used for stylistic CSS and responsive UI. In the server side backend logic is implemented using python 3.10+ using FastAPI framework, which efficiently handles asynchronous API requests. For fundamental intelligent processing, the system uses the sentence-transformers library (e.g. MiniLM-L6-v2) for generating vector embeddings and ffmpeg for accurately video processing and video clips. Data persistence and high-speed vector similarity search is handled by Supabase, which is a hosted PostgreSQL database with the pgvector extension. Additionally, Visual Studio Code is used as the primary Integrated Development Environment (IDE) and Git is used for version control. A thorough list of these software tools, and languages, are given in:

Table 1.2: Software Requirements

| Category                     | Tool / Language                  | Purpose                                           |
|------------------------------|----------------------------------|---------------------------------------------------|
| <b>Frontend Framework</b>    | React (with Vite)                | Building the interactive user interface.          |
| <b>Programming Languages</b> | TypeScript, Python 3.10+         | Frontend logic and Backend server implementation. |
| <b>Backend Framework</b>     | FastAPI                          | Creating high-performance API endpoints.          |
| <b>Database</b>              | Supabase (PostgreSQL + pgvector) | Storing relational data and vector embeddings.    |
| <b>ML &amp; Processing</b>   | sentence-transformers, ffmpeg    | Generating embeddings and processing video files. |
| <b>Development</b>           | Visual Studio Code, Git          | Code editing and version control                  |

### 1.5.3 Data Requirement

The implementation of NeuroClip depends on specific datasets for the training and validation of the basic AI components. The requirements of data are classified as follows:

#### Operational Input Data:

**Video Repository:** A diverse set of long-form videos (MP4, MOV, MKV) which include lectures so that we can have tests with the end-to-end pipeline.

**Derived Artifacts:** Automatically generated a set of transcripts in the context of this task, in which the transcripts are stored with timestamps in the vector database to obtain 384-d vector embeddings for semantic retrieval.

#### Model Validation Datasets:

**Semantic Search:** A custom validation initialization, in which video transcripts and natural language questions are used to evaluate the relevance of the clips found by the all-MiniLM-L6-v2 model for embedding.

## 1.6 Report Organization

This report is divided into a number of chapters. Chapter 1 (Introduction) addresses the motivation of the project, problem statement, objectives, scope, and resource requirements of the project. Chapter 2 (Background and Existing Work) provides related literature, existing systems and a summary of their limitations, which establishes the necessity of NeuroClip. Chapter 3 (System Requirement and Specifications) describes the functional

and non-functional requirements, as well as modular design. Partly included within Chapter 4 (System Modeling and Design) will use the use of diagrams to illustrate the architecture and flow of the system.

**CHAPTER 2**

**BACKGROUND AND EXISTING**

**SYSTEMS**

This chapter discusses the current solutions and related research in the area of video processing, summarization, privacy and compression in order to determine the need and novelty of the NeuroClip system.

The current problem is the use of different and separate tools for the various video processing requirements. Many solutions that exist today do not have automatic detection capabilities, have trouble with real-time performance or sacrifice quality for file size. For example, manual editing takes a lot of time., existing summarization tools fail to capture important context or are restricted to particular platforms such as YouTube.

## **2.1 Related Literature Review**

Reimers and Gurevych (2019) proposed an adaptation of pretrained BERT network into an siamese and triplet network called Sentence-BERT (SBERT), which is used to obtain semantically meaningful sentence embeddings. The article tries to deal with the enormous computational cost of doing clustering and semantic search with raw BERT, with the fastest time to find the most similar pair in a set of 10,000 sentences being almost 65 hours. Their results proved that SBERT was capable of decreasing this inference time to about 5 seconds, and still high accuracy on semantic textual similarity (STS) benchmarks. A drawback that can be seen in this work, however, is that although SBERT is very efficient in first-stage retrieval (Bi-Encoder), it sometimes does not have the fine-grained retrieval characteristic of full Cross-Encoder architectures when dealing with complex, nuanced queries in which the interaction between the query and document has to be deep.

Nogueira and Cho (2019) studied the use of BERT in Passage Re-ranking with BERT and effectively used the Cross-Encoder architecture to enhance the precision of retrieval. They suggested in their work that the query and the candidate passage be fed together into the Transformer model to predict a relevance score, and through which the self-attention mechanism can model the interaction between query terms and document contents entirely. The results showed that this technique has the highest possible state of art even on such datasets as MS MARCO and it has much better results on Mean Average Precision (MAP) as compared to traditional BM25 and vector-only methods. The main weakness that has been pointed out in this paper is the large latency of computations; since the query and document are to be run simultaneously, the system cannot afford to pre-compress the representational of the documents, and thus, the system is prohibitively slow to run on large collections, without a filtering step beforehand.

The landmark paper was Vaswani et al. (2017) titled Attention Is All You Need that introduced the Transformer architecture, which was purely based on the attention mechanisms, without recurrence or convolution altogether. The article is important because it forms the foundation of modern ASR (Automated Speech Recognition) and embedding models, applied in video transcription. The article established that the Transformer can be much more parallelized and achieves new state-of-the-art outputs in the quality of translation. Nevertheless, the original Transformer architecture has an inherent drawback, which is a quadratic complexity in computation with the input sequence length that becomes difficult to handle when dealing with very long video transcripts or documents and requires methods such as windowing or segmentation to deal with memory and performance limits.

Table 2.1: Summary of Reviewed Literature

| Year | Paper Title        | Contribution                                                    | Techniques                          | Limitations                                                                                                                 |
|------|--------------------|-----------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 2019 | Reimers & Gurevych | Introduced SBERT for efficient semantic similarity search.      | Siamese BERT-Networks (Bi-Encoders) | Embeddings may lack the fine-grained precision of Cross-Encoders for complex query-document interactions.                   |
| 2019 | Nogueira & Cho     | Demonstrated high-precision passage reranking.                  | Cross-Encoder BERT Architecture     | High computational latency makes it unsuitable for real-time retrieval on large datasets without pre-filtering.             |
| 2017 | Vaswani et al.     | Proposed the Transformer architecture enabling parallelization. | Self-Attention Mechanisms           | Quadratic complexity regarding sequence length makes processing very long inputs (like full video logs) resource-intensive. |

## 2.2 Related Systems

The current commercial applications and research systems, as outlined in the summary table below have identified some major limitations that NeuroClip seeks to overcome. Most well-known video editors such as CapCut and Inshot are based on manual blurring options



and are not able to detect faces automatically. Specialized programs such as Summarify can provide fast AI-generated summaries but usually are restricted to particular content, such as YouTube videos, and are not intended to be used to process user-posted files in real-time.

Table 2.2: Summary of Existing System

| <b>Sr No.</b> | <b>Article No / APP/ System</b>                      | <b>Year</b> | <b>Features</b>                                                                                | <b>Strength</b>                                    | <b>Limitations</b>                                                   |
|---------------|------------------------------------------------------|-------------|------------------------------------------------------------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------|
| 1             | CapCut                                               | 2024        | Video editing with manual face blurring                                                        | Fast processing; allows manual face blurring       | Cannot auto-detect faces for blurring                                |
| 2             | Easy Mosaic                                          | 2022        | Hard blur effects, real-time blur preview                                                      | Strong blur effect; allows preview before applying | Slow tracking; lacks automatic face detection                        |
| 3             | Summarify                                            | 2021        | AI-powered summarization of YouTube videos via key frame extraction                            | Quick and effective summarization of long videos   | Limited to YouTube content; not designed for real-time processing    |
| 4             | Sensitive Information Detection in Videos Using CNNs | 2021        | Employs convolutional neural networks to detect and blur sensitive information in video frames | High accuracy in privacy protection                | Decreased performance in complex scenes or with occlusions           |
| 5             | Real-Time Video Compression using Neural Networks    | 2020        | Implements a neural network approach to compress video data in real time                       | Balances compression rate and quality              | Requires GPU acceleration; may underperform on low-resource hardware |
| 6             | Notta                                                | 2020        | Automated speech transcription with video support; summarization and note-taking               | Accurate speech recognition and summary generation | Limited language support; struggles with noisy audio                 |

## **2.3 Identified Problem from Existing Work**

A critical analysis of the reviewed literature and existing video search applications shows that there are significant limitations in both retrieval accuracy and computation efficiency. Existing commercial platforms are largely based on brittle keyword matching techniques that do not account for the semantic nuance of user queries, and this leads to irrelevant or broken up segments of video with necessary context. Technically, as we have seen in the case of architectures such as BERT and Cross-Encoders, there is always a clear trade-off between high-precision models, which have prohibitive latency, and faster models of the Bi-Encoder type, which sometimes do not capture subtle interactions between the queries and documents. Furthermore, from a non-functional point of view, the current workflows are highly inefficient; the current systems lack proper caching mechanisms for the embeddings, requiring the systems to re-process long form videos redundantly for each new search, which results in unacceptable delays and resource wastage. NeuroClip fills these research gaps by implementing a hybrid search pipeline combining the speed of pre-computed embeddings and the precision of reranking, and introducing a stable job-based caching architecture. This way, videos will be processed once and queried immediately, which is an effective balance between high semantic accuracy and system responsiveness.

## **2.4 Selected Boundary for Proposed Solution**

To overcome the lack of semantic context and computational inefficiency identified in Section 2.3, NeuroClip will use a semantic search engine that can understand the natural language queries via sentence level embeddings. The system will include an automated pipeline for video transcription, video vectorization and windowed retrieval to guarantee context-aware results as well as persistent storage mechanism to cache these artifacts using unique job ID to avoid redundant processing and support instant retrieval for subsequent queries.

On the other hand, the proposed system will not be able to support streaming video analysis in real-time, due to the latency that currently exists in high-precision reranking models, specifically focusing on the batch processing of the uploaded files. Additionally, the project scope clearly excludes the ability to train proprietary Automatic Speech Recognition (ASR) models and instead uses proven pre-trained libraries and will not provide advanced non-linear video editing capabilities like transitions or visual effects but will be limited to the ability to accurately extract a clip and simple playback.

**CHAPTER 3**

**SYSTEM REQUIREMENT AND  
SPECIFICATIONS**

Contemporary digital world is characterized by a huge amount of video content, which makes it a primary medium of communication, entertainment and learning. This chapter outlines the complete requirements and specifications for the successful development and implementation of the NeuroClip application. It includes the technical system specifications and establishes the architecture of the system software and its operating environment and defines the various modules of functionality that drive the basic logic of the system. Furthermore, it lays out the critical non-functional requirements such as usability, performance, reliability and security which are essential for delivering a seamless and effective user experience across the Progressive Web Application platform. The following sections contain a systematic breakdown of the design of the system and required functionality

### **3.1 System Specification**

System specifications serve as the technical blueprint for the application, defining the hardware, software, and operational parameters required for its functionality. The NeuroClip system will be built as a cross-platform Progressive Web Application (PWA) using React for the front end, integrated with a Python/FastAPI backend to handle the intensive tasks. Data storage will utilize PostgreSQL for metadata and a cloud storage solution for the videos. This architecture characterizes a high-performance system designed for cross-platform accessibility, as detailed in the Resource Requirement section.

### **3.2 System Modules**

System modules represent different functional units of the software architecture, which are responsible for different tasks such as data processing, user interaction or storage management. NeuroClip has been architected into two main modules to streamline the computational heavy-lifting part of video processing out of the interactive logic part of the search and retrieval part. This holds true in the modular design, which ensures that it is scalable, maintainable and allows to update certain elements such as the model of embedding without disrupting the entire system.

Each of the modules is described as follows:

#### **3.2.1 User Registration & Login**

This module handles the security and identity verification of the system with the help of Supabase Auth. Instead of solely using the social login from the third-party, the system provides a robust flow of email-and-password authentication. It has a compulsory email

verification process in order to make sure that the user is genuine and there are no spam accounts. Additionally, a secure "Forgot Password" mechanism is added that can be used by users for account recovery through email emails. This module ensures that any video data, processing histories, are strictly isolated and cannot be accessed by any authenticated owner.

### **3.2.2 Video Upload & Management**

The main way of ingesting data is the Video Upload & Management module. It gives functionality to the user to upload video file from their local devices to the cloud storage. Before processing, we have added a trimming feature in this module in which a user could select and crop specific portions of a video they would like to analyze instead of processing entire files. Additionally, it contains a dashboard feature that fetches and presents a list of previously uploaded and processed videos so that the users can manage them, replay them, or delete the history efficiently.

### **3.2.3 Video Summarization**

This is the core processing module of the system, which is driven by Vector Embeddings. As opposed to traditional frequency-based summarization, this module is based on deep learning models (such as Sentence-Transformers) that can turn video transcripts and visual context into high-dimensional vectors. By analyzing these embeddings, the system determines semantically important clusters and nuanced moments of the video that represent the essential topics of the video. It has these segments to be scored on the basis of their semantic density and their relevance to the intent of the user. High scoring parts are then mathematically selected and stitched together to produce a brief coherent summary of the original with the context preserved.

### **3.2.4 Video Summarization in Text**

To complement the visual summary, a textual explanation of the video content is created in this module. Making use of the identical concept of the embedding, it then groups together related sentence vectors to abstract the video's narrative. It then uses Natural Language Processing (NLP) to produce a structured summary of the text in the form of timestamped bullet points to the user. This means that people can easily scan through the key points of the video and jump to particular timestamps, without having to watch the entire video.

### **3.2.5 Sensitive Information Blurring**

To overcome privacy concerns that are ingrained in public video data, this module plays

out as an automated redaction system. Leveraging the YOLOv8 object detection model, it looks at frames of video on a frame-by-frame basis to detect sensitive objects such as human faces, national ID cards and vehicle license plates. Once these objects have been detected the system applies a dynamic blurring filter specifically to these regions of interest (ROI). This guarantees that the identities of people and private information isn't disclosed in the end result and makes the video safe to be distributed more broadly.

### **3.2.6 Video Compression**

This module optimizes the storage and transmission of the processed videos. After all summarization and blurring are done, this system uses ffmpeg to re-encode the video file. It uses efficient compression algorithms (like H.264/H.265) to compress the file size by some 75% while using minimum visual noticeable quality. This reduction drastically reduces the amount of bandwidth consumed and storage costs as it allows for the final summaries to be lightweight and easy to share across a variety of different platforms with no noticeable loss in video quality.

## **3.3 Functional Requirements**

Functional requirements specify the specific behaviours and functions of the system, specifying what the system should do to satisfy user requirements. For NeuroClip these requirements are centered around the holistic ingestion, processing and retrieval of video content. The system is designed to allow its users to have a robust platform for uploading videos, creating AI-based summaries, providing privacy through automatic blurring, and retrieving content through semantic search. The basic functional requirements include authentication, video management, intelligent summarization, and safe playback.

Each of the key functional requirements is described in detail below:

### **3.3.1 User Registration and Authentication**

The system shall provide mean of new user account registration using valid email address, and password. It needs to implement a verification process in which a confirmation link is sent to the user's e-mail to verify their identity before their account is activated. Additionally, the system needs to have a secure login option for returning users and a "Forgot Password" option for account recovery.

### **3.3.2 Video Upload and Preprocessing**

The system shall allow authenticated users to upload video files (in file formats such as .mp4, .mkv) to the cloud storage. During the uploading process, the system should enable

the several web users to choose to trim the video in order to select the specific segment that they want to be processed. Once uploaded, the system shall automatically launch a background job to verify the file format and size and queue it to the transcription and generation of the embeddings.

### **3.3.3 Automated Video Summarization**

The system shall automatically create a brief presentation of the uploaded video. This feature has to leverage deep learning models to detect important moments which are based on semantic importance and queries defined by users. The system shall extract relevant video segments, and stitch them together to create a coherent video summary clip containing the essential information from the original video.

### **3.3.4 Semantic Search and Retrieval**

The system shall provide search interface with natural language queries from the user. It has to use the vector embeddings to be able to semantically match the query that the user is typing with the transcripts of the videos. The system shall provide a list of available relevant video clips based on their semantic similarity score, so that users can find specific information without exact matches of relevant keywords.

### **3.3.5 Sensitive Information Redaction**

The system shall automatically detect sensitive visual information of video frames, namely human faces, license plate, and ID card. Upon its detection, the system has to apply a blurring filter on these areas in order to uphold the privacy of people. This process needs to be automated and applied to the final output video before they are available for viewing or sharing.

### **3.3.6 Video Compression and Playback**

The system shall compress processed videos in order to reduce the file size at acceptable visual quality for streaming. It must support an inbuilt video player that gives users the ability to stream the automatically generated summaries and retrieved clips directly within the web application. The player must support the standard playback controls (i.e. play, pause, volume, fullscreen) and the player must be able to load the content efficiently, to make sure of a good user experience.

### **3.4 Non-Functional Requirements**

Non-functional requirements or NFR defines quality attributes, constraints, and performance standards of the system, not necessarily the behaviors. While functional requirements are what the system does, non-functional requirements are how good a job the system has to do. These requirements are crucial in the process of ensuring the user experience, scalability and long term viability of NeuroClip. The system focuses on Usability, Reliability, Performance, Security and Scalability to meet the processing of high volume video data requirements.

#### **3.4.1 Usability**

An appropriate question is "how intuitive and easy is it for the user to learn and use the system". Measuring this is usability. Given that NeuroClip is aimed at a wide audience, including those that may not be technically skilled, the interface needs to be a simple one, which they can navigate around.

Measurement: A new user should be able to upload a video and create a summary within two takes without having to go outside of the system to obtain documentation. The interface for cutting videos and reading summaries should be self-explanatory by using common icons and labels.

#### **3.4.2 Reliability & Availability**

Reliability guarantees that the system works properly and consistently over the years without breaking down when errors occur.

Rationale: Video processing is resource intensive; failures during long uploads/processing job is frustrating.

Measurement: Uptime during operational hours of the system should be 99.5% In the event of processing failure (e.g. a corrupted video file), the system does not crash but instead withdraws a readable and comprehensible error message and allows the user to retry the operation with no data loss.

#### **3.4.3 Performance & Reactiveness**

Performance is the responsiveness and the speed of the system under expected load conditions.



Rationale: Users of the site can expect immediate (to near-immediate) feedback for search queries and reasonable processing times for video uploads.

Measurement:

Search Latency: It is the rule of thumb that semantic search queries should take less than 2 seconds to complete

Processing Speed: For a standard 10-minute video the full pipeline (transcription, embedding) should be done in a time of 1 minute on the target hardware.

UI Responsiveness: The Web interface should be able to load and react to user interactions (click, navigation) under 200ms.

#### **3.4.4 Security & Privacy**

Security specifies the security of user data, making it discrete, undamaged and available.

Rationale: NeuroClip does process potentially sensitive personal videos; the privacy of the information is paramount.

Measurement:

Data Isolation: User data (videos, transcripts) needs to be strictly isolated using Row Level Security (RLS) policies; there should be no way for a user to access the content of another user via manipulations to API.

Encryption: All the data in transit should be encrypted over https/tls.

Access Control: The system needs to use authentication to check anyone's access to any API endpoint; if any unverified request is received by the system, they should reject the request by sending 401 Unauthorized status.

#### **3.4.5 Portability & Compatibility**

Scalability is defined as the capability of the system to manage the feeling workload by adding resources.

Rationale: As the base of users increases and more videos are uploaded, the database as well as the processing pipeline must not be bottlenecks.

Measurement: The system architecture should be able to support 50% more concurrent users without exceeding 3 seconds for the search latency.

# **CHAPTER 4**

## **SYSTEM MODELING AND DESIGN**

System modeling and design is a very crucial part of the software development process where the requirements specifications are translated into the construction blueprint of software. It involves the crafting of abstract representations of the system to understand its structure and behavior and its interactions before it is actually implemented. This phase is essential for identifying potential design flaws, ensures that the system scales and makes sure that architecture of the system is clear for all stakeholders. By using the various standard modelling diagrams, developers can visualize the data flow and user interactions, as well as the internal logic, and this can help to create a more robust and efficient development process.

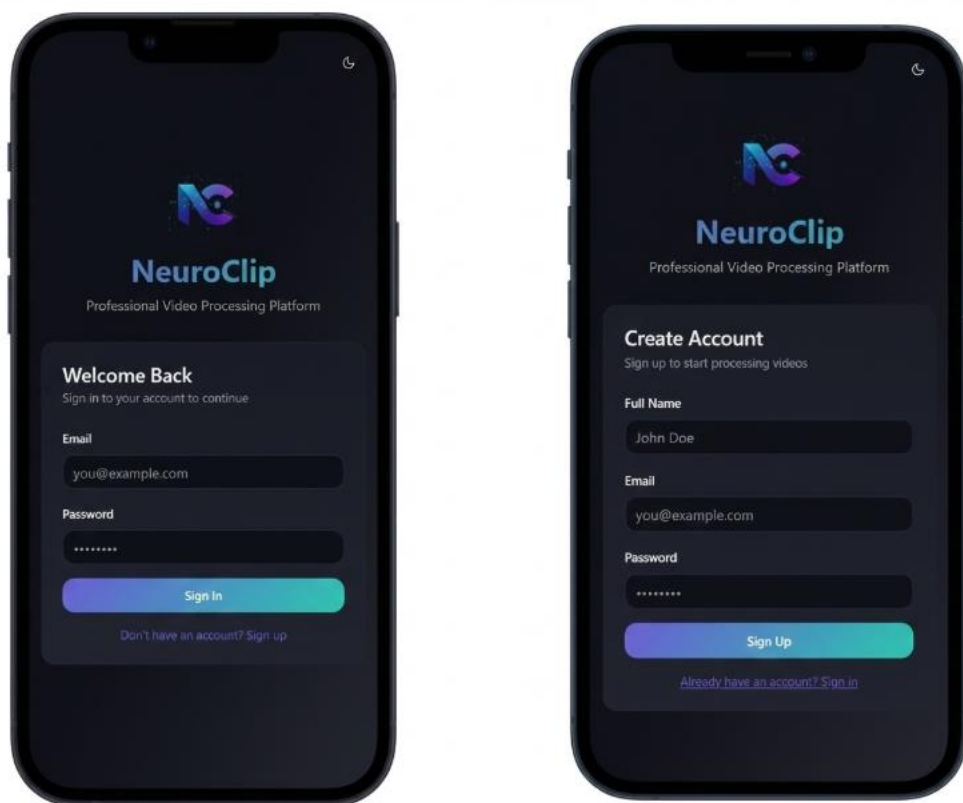
The system being discussed, NeuroClip, is an intelligent video processing system that aims at improving content retrieval using AI-driven summarization and semantic search. It integrates complex components like automated transcription, deep learning-based embedding generation and object detection for privacy protection. In order to properly model this system, a number of Unified Modeling Language (UML) and structured diagrams have been applied. These include Use Case Diagrams to show user interactions, Activity Diagrams to show logic flow, Data Flow Diagram (DFD) to show information flow, and Sequence Diagrams to show the working of objects at a specific time. Additionally, architectural diagrams also give you a high-level view of frontend, backend and database integration.

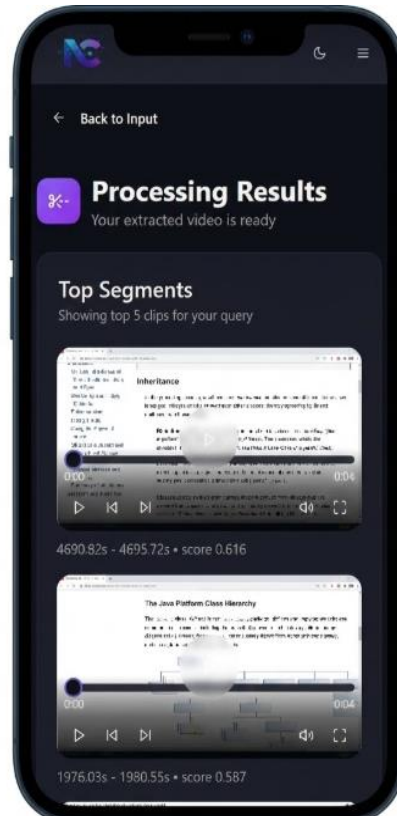
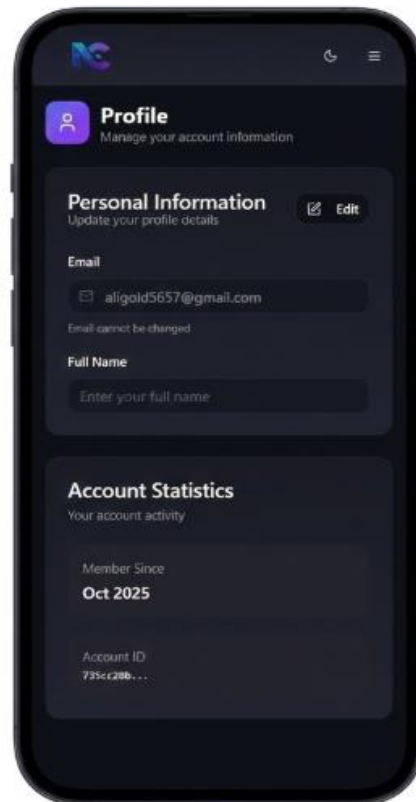
This chapter is structured so that it helps obtain a big picture of the design of the system. It starts with the System Design and Analysis followed by detailed behavioral models such as Use Case Diagrams and Activity Diagrams. Data-oriented models such as DFDs are then presented followed by interaction models such as Sequence and System Sequence Diagrams. Finally, the structure is presented with detailed information using Class Diagrams and Architectural layouts.

## **4.1 System Design and Analysis**

System design and analysis is the central connecting link between requirements engineering and software construction, that is, between functional specification and a powerful and scalable technical blueprint. For NeuroClip, this phase outlined a modular full-stack architecture consisting of React, a frontend for user interaction, FastAPI, a backend for core logic and AI processing and Supabase (PostgreSQL) database for efficient data and vector storage. For the precise modeling of the static setup and dynamic behaviour

of the system in question, standard UML diagrams were kept, which include Use Case Diagrams for mapping the interactions to the Data Flow Diagrams (DFD) for the visualization of the video processing pipeline and the Sequence and Class Diagrams for a detailed description of the service communication and data structure, which are collectively responsible for a comprehensive understanding of the operations before implementation.





## 4.2 Use Case Diagrams

A Use Case diagram is a behavioral diagram in the Unified Modeling Language UML for representing the dynamic interactions between external entities, called actors, and the system being developed. It is a visual blueprint of functional requirements of the system thereby defining the boundaries and scope of the system. For NeuroClip the use case diagram is important because it is a model for the user centered functionalities of the NeuroClip platform showing how the end user can start complex AI workflows (video ingestion, semantic search, etc.) by simple functions of the interface.

The currently available use case diagram for NeuroClip shows the "End User" as the main actor who interacts with the system to execute the fundamental interactions. These tasks include being able to securely login, upload video files to be processed, submit natural language inquiries for the retrieval of specific clips, and view processing history. The diagram emphasizes the point to point dependencies between the user actions and the backend responses of the system, such as the generation of transcripts and the computation of embeddings and thus offers a high-level understanding of the capabilities of the system from the user's perspective (as shown in Figure 4.1).

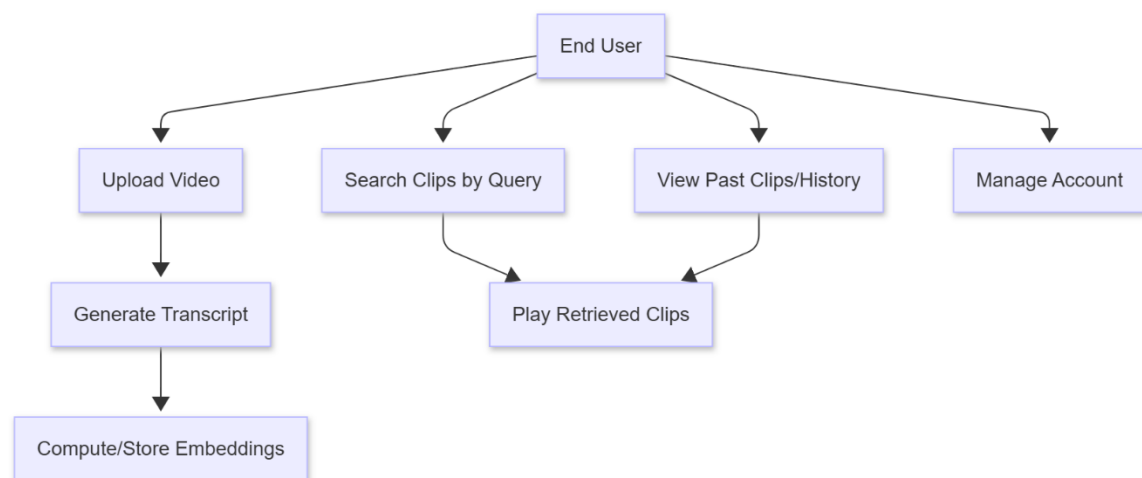


Figure 4.1: Main Use Case Diagram

## 4.3 Detailed Use Case

Full dress use cases are very detailed descriptions of the system's behavior in a given interaction. Unlike completely brief use situation diagram, that provides an overview of relationships, complete dress use cases specify the preconditions, post-conditions, the primary success situation (happy path), and alternative flows (error handling) for a specific

function. They are very important to bridge the gap between requirements and implementation in order to make sure that developers know the exact logic needed to implement some complex processes and to get a base to design the test cases.

For NeuroClip, we have chosen "Upload Video", "Search Clips by Query" use cases to be detailed in the form of full dresses. The reason to choose this two use cases are to encapsulate the fundamental complexity of the system:

- **Upload Video:** This is not a simple file transfer though, it is used to start an asynchronous pipeline of transcription, embedding computation, and sensitive information blurring.
- **Search Clips by Query:** This involves complex logic comprising semantic vector retrieval, reranking and dynamic video segmentation using ffmpeg.
- **Manage Videos:** This controls the information of the user about the videos including video trimming and deleting.

Other simple use cases such as "Login/Register" or "Manage Account" are standard web practices and thus have no complex full dress descriptions specified in this report.

#### 4.3.1 Search and Clip Merge - Full Dress Use Case

This use case describes the onboarding process in which a user sets off his or her cessation journey. It is used in that it defines the basis data (quit date, consumption habits) that plays the role of feeding the rest of the algorithms that personalize the application. The detailed specification is given in the Table 4.

Table 4.1: Full Dress Use Case

| Use Case Selection | Comment                                                   |
|--------------------|-----------------------------------------------------------|
| Property           | Description                                               |
| Use Case Name      | Search and Clip Merge                                     |
| Scope              | NeuroClip System (Frontend, Backend, Database, AI Models) |
| Level              | User Goal                                                 |
| Primary Actor      | Authenticated User                                        |

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Stakeholders and Interests</b> | <p><b>User:</b> Wants to retrieve specific information from long videos without manual searching.</p> <p><b>System:</b> Needs to execute resource-intensive tasks (Search, ffmpeg) efficiently.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Pre-conditions</b>             | <ol style="list-style-type: none"> <li>1. User is logged in and has a valid session.</li> <li>2. The target video has been uploaded and processed.</li> <li>3. Transcript and sentence embeddings are generated and stored in the database linked by <code>job_id</code>.</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Success Guarantee</b>          | The system displays a list of video clips that semantically match the user's query, including the start/end times and the specific video segment ready for playback.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Main Success Scenario</b>      | <ol style="list-style-type: none"> <li>1. User navigates to the <b>VideoClips</b> page for a specific <code>job_id</code>.</li> <li>2. User configures retrieval parameters (Window Size, Stride, Min/Max Duration).</li> <li>3. User enters a natural language query (e.g., "explain password security").</li> <li>4. User clicks "Search".</li> <li>5. System retrieves sentence embeddings associated with the <code>job_id</code>.</li> <li>6. System performs windowed semantic search and computes similarity scores.</li> <li>7. System reranks the top results using the Cross-Encoder (if enabled).</li> <li>8. System merges adjacent relevant windows to form coherent segments.</li> <li>9. System generates the physical video clip using <code>ffmpeg</code>.</li> <li>10. System returns the clip URL and aggregated text summary.</li> <li>11. User plays the retrieved clip.</li> </ol> |



|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Extensions</b> | <p><b>3a. Invalid Query:</b></p> <ol style="list-style-type: none"> <li>1. System detects empty or invalid query string.</li> <li>2. System displays a warning toast to the user.</li> </ol> <p><b>7a. Reranker Model Unavailable:</b></p> <ol style="list-style-type: none"> <li>1. System logs the error.</li> <li>2. System falls back to standard embedding-only ranking.</li> <li>3. System proceeds to Step 8.</li> </ol> <p><b>8a. No Matching Windows Found:</b></p> <ol style="list-style-type: none"> <li>1. System determines that no segments meet the similarity threshold.</li> <li>2. System returns an empty result set.</li> <li>3. System suggests relaxing parameters (e.g., smaller window size, larger max duration).</li> </ol> |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### 4.4 Activity diagram:

An activity diagram is a behavioral diagram in the Unified Modeling Language (UML) that is used to visually depict the flow of control or data from one activity to another activity. It is basically just like a kind of a sophisticated flowchart that is used to model the dynamic part of a system. Activity diagrams have meaning in modelling the workflow of a particular business process, a high-level account of the functionalities within the system and investigating the business requirements mapping out the sequence of actions and decisions and the concurrent threads.

For NeuroClip the activity diagram is prepared from the viewpoint of the end to end user process to convert a raw video to a searchable asset by sequential steps. The process starts with user authentication (Registration and Email Verification) or Login, then there is the video uploading phase. The diagram illustrates the internal logic of the system, including decision points such as checking the format of the file and checking if the transcript was generated successfully. If these checks pass then the flow continues to the embedding

computation and enables the user to do semantic searches. Finally, the activity finishes by taking the user to the fetched clips or storing the processing history. The graphical representation of such a workflow is presented in Figure 2.

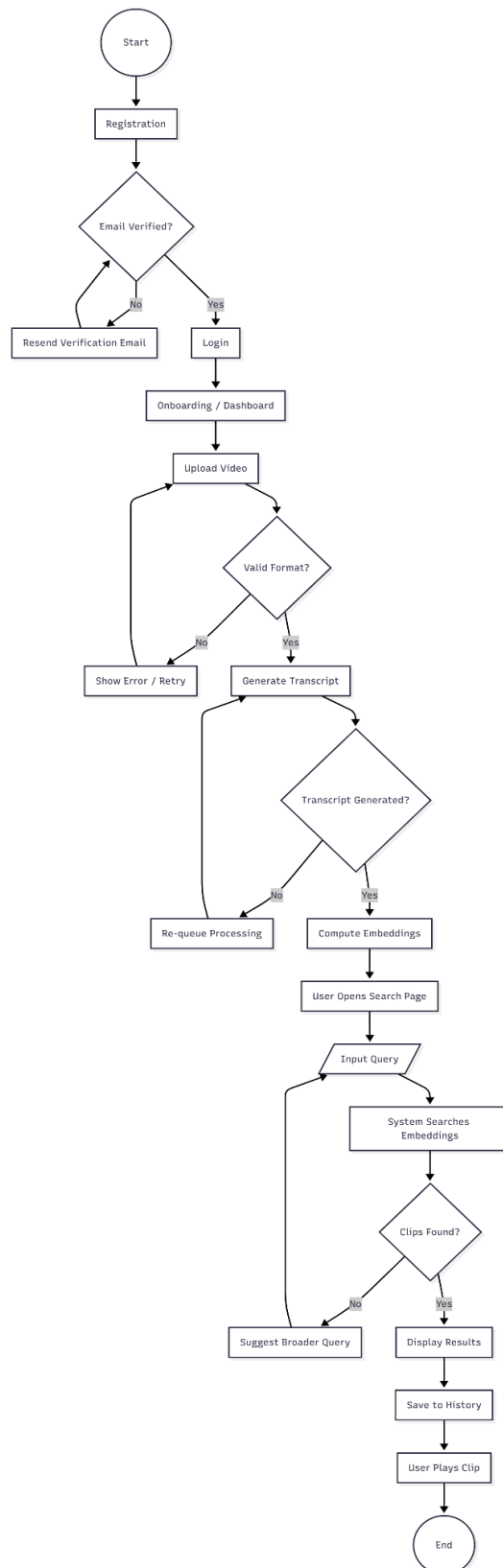


Figure 4.2: Activity Diagram

## 4.5 Data Flow Diagram:

A Data Flow Diagram or DFD can be defined as a graphical depiction of the data flow through an information system. It is a way of modeling the flow of data entering the system, how it is processed and where it is stored, but without the presentation of any of the control logic, such as loops and decisions. DFDs are very important for NeuroClip, as they provide a clear understanding of the process of converting raw video inputs into searchable knowledge assets (transcripts, embeddings, clips). They help to identify the source of data (Users), processes that convert data (Transcription, Embedding) and data destination (Storage, User Interface).

For this project, we have created a Context Diagram (Level 0) depicting the external boundary of the system and Level 1 DFD depicting the internal processing modules.

### 4.5.1 DFD Level 0 (Context Diagram)

The Level 0 DFD or Context Diagram is the high-level view of the entire NeuroClip system as a single process interacting with other external entities. It depicts the "End User" sending the system video files and search queries and receiving processed clips and summaries in return. All the internal details are abstracted and the focus is on the boundary of the system and the external data exchanges. The Level 0 DFD is given in Figure 4.3.

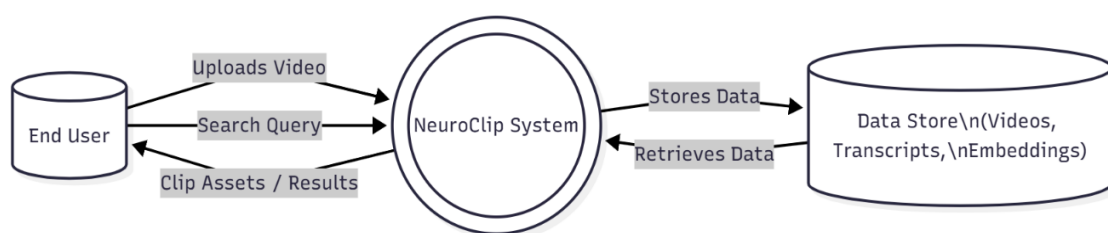


Figure 4.3: DFD Level 0

### 4.5.2 DFD Level 1

The Level 1 DFD details the one process from Level 0 into significant sub-processes. For NeuroClip this is expanded to give the diagram:

Auth & Profile Management: User authentication & profile management.

Video Ingestion: Ingesting video as raw video and creating transcripts of it.

Embedding Computation: Transforming Text to Vector.

Clip Search & Assembly: Query processing and relevant video retrieval.

At this level are revealed the internal data stores (Videos, Transcripts, Embeddings, History) and the movement of the data between the stores and the processing modules. The Level 1 DFD can be seen in Figure 4.4.

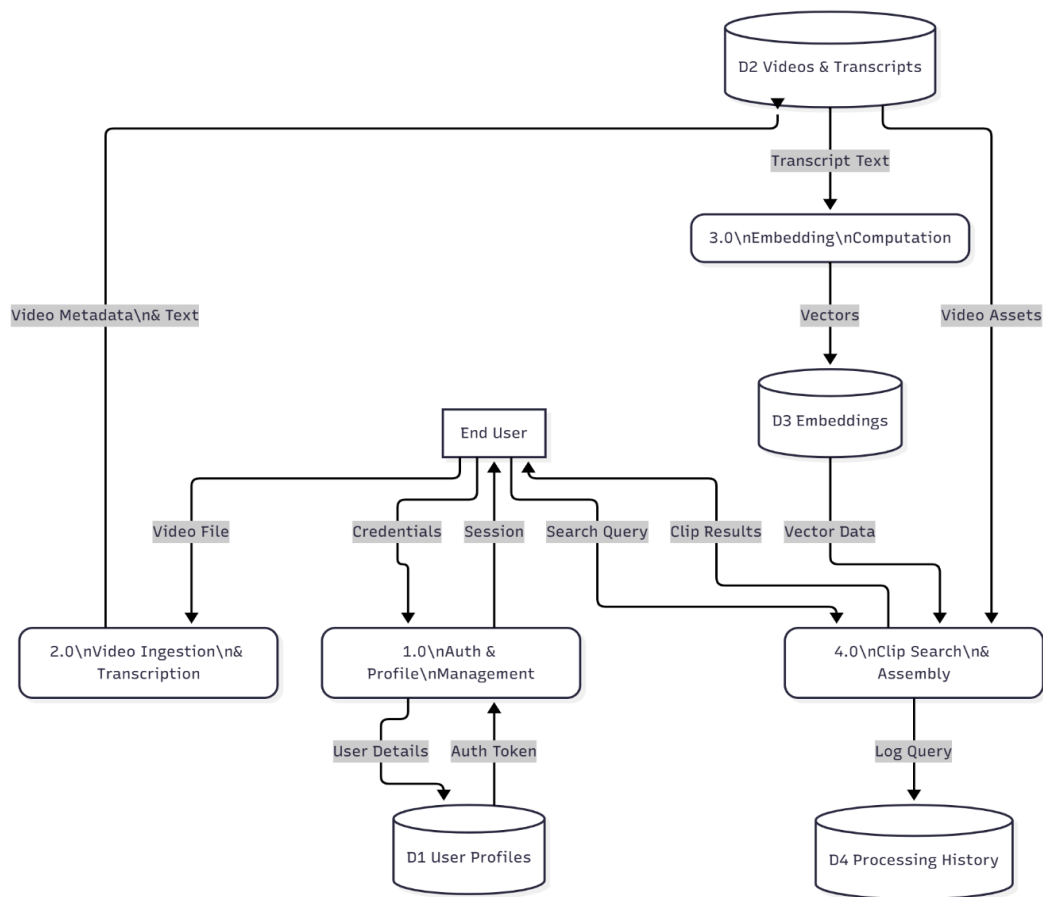


Figure 4.4: DFD Level 1

## 4.6 System Sequence Diagram

System Sequence Diagram - A System Sequence Diagram (SSD) is a behavioral diagram which shows an interaction sequence between an external actor (the user) and the system (a "black box"). This records the input events that are generated by the actor, the order in which they occurred and the output responses the system makes. SSDs are important for NeuroClip as they mediate between the use case and detailed design. By concentrating

exclusively on the input/output events for certain cases (such as searching for a clip or uploading a video), SSDs define the interface contracts for the system before going into details of the object interactions within the system.

We have created an SSD for the scenario of "Clip Search Flow". This scenario was selected because it is the fundamental value of the application, a request from a user with a natural language question and the actionable response with videos being returned to the user. The SSD represents the visualization of the user making a search request (with param(s) [text, window size]), of the system validating the session, of the processing of the query against the vector database, and finally of the return of the user with the return of the JSon payload containing the playable clip URLs. The System Sequence Diagram is given in Figure 4.5.

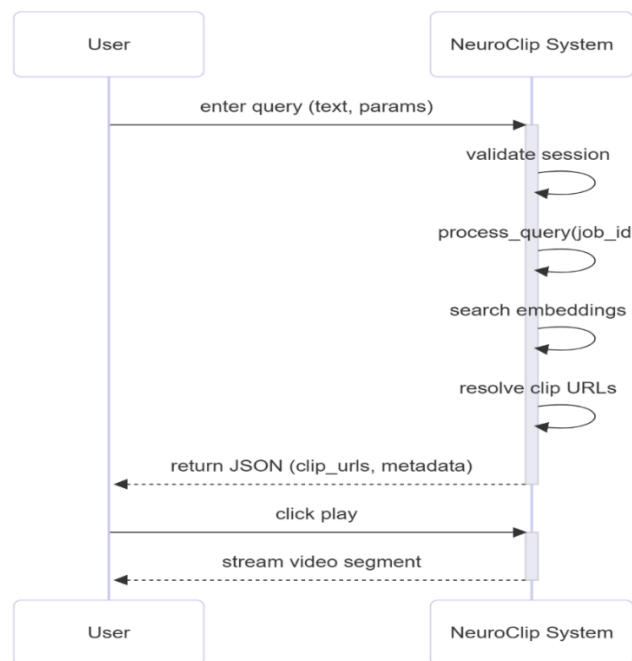


Figure 4.5: System Sequence Diagram

## 4.7 Sequence Diagram

A Sequence Diagram (SD) is an interaction diagram that describes how operations are performed by specifying the interaction between objects in the context of a collaboration. Unlike the System Sequence Diagram which takes the system as a black box, the Sequence Diagram opens up the black box to see how the objects inside the system (Services, Database, Frontend components) will communicate to fulfill a request. It is time focused, thinking about the sequence of messages exchanged between these objects, along a vertical

timeline.

For NeuroClip has created the Sequence Diagram to model the Email Verification and Redirect scenario. This flow is technically critical since there is coordination between the User, the Authentication Service (Supabase Auth), an external Email Provider, and the Frontend application logic. The following diagram shows the specific sequence of events: The user decides to sign up, the Auth service sends the user a verification email, the user clicks the confirmation link, the Auth service sends the user back to the application so that the user can have a secure session and route them to the onboarding dashboard. This detailed view ensures that the security handshakes are implemented. The Sequence Diagram is as shown in Figure 4.6.

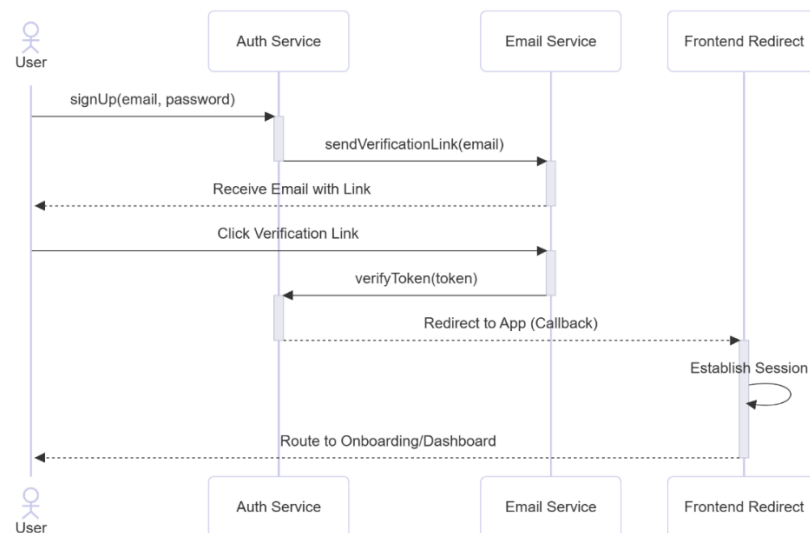


Figure 4.6: Sequence Diagram

## 4.8 Design Class Diagram

The Design class diagram (DCD) is a static structure diagram in the Unified Modeling Language (UML) that describes a system's structure using the system's classes and their attributes and operations (also known as methods) and how they are related to each other as objects. The DCD, unlike the domain model that is focused on conceptual entities, represents the actual specification of the software construction, making the connection between design and code construction. For NeuroClip, it is the Class Diagram, which defines the schema for the entities of the database (User, Video, Embeddings) and the

structure of the back services (Search, Processing) simply manipulating this data.

The Class Diagram for NeuroClip centers around the User class which stores information about the user's authentication and has a one-to-many link with the Video class, indicating that one user can upload many videos. The Video class is the core of the system with attributes such as videourl and duration and consists of multiple TranscriptSegment and EmbeddingRow objects, which represent the texts and vectors respectively. Additionally, the diagram shows classes like SearchService with its methods like processquery() and rerankresults() and ClipGenerator which is responsible for the generateclip() operation. The interaction and structure of these classes is shown in Figure 4.7.

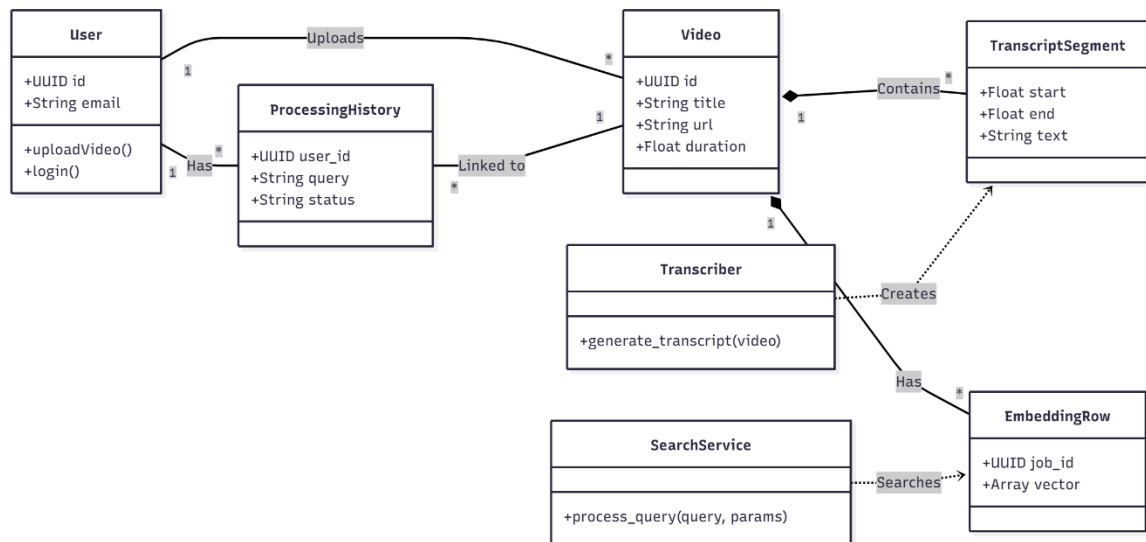


Figure 4.7: Class Diagram

## 4.9 Architectural diagrams

Software architecture can be seen as the original blueprint for a system to help establish a high-level view of a system to control its complexity and to define the required communication and coordination mechanisms between different parts of the system. It defines the structural organization or arrangement of the software elements and their relative relationships to each other with the intention of ensuring, that the system satisfies its functional and performance requirements.

For NeuroClip, we have created some specific architectural diagrams to show various views of the system. Given the full stack nature of the application (React Frontend, FastAPI



Backend with Database), we have paid attention in the following diagrams:

Interface Design Diagram: To show the flow of the interaction with the user and the Hierarchy visual components

Component Level Design: To draw the modular structure of the Back End services, frontend libraries.

Deployment Diagram: To represent the physical distribution of the software artifacts on the hardware nodes (Cloud/Local environment).

These diagrams were chosen to give a full picture of how the user is interacting with the system, how the code is organized and how the system is hosted.

#### **4.9.1 Interface Design**

While interface design deals with the visual aspects, and the mechanisms that are provided to the user for interaction with the application. It is a must for NeuroClip as the system is highly dependent on user interaction for all processes such as uploading videos, trimming content and natural language queries. The interface design diagram shows the relationship between various web pages and the reusable components (e.g. the Header, Video Player and Search Bar) included in them.

The design is hierarchically organised in which the App container handles the routing. Some important interfaces are Authentication Pages (Login/Register) that prevent access to the system. The Dashboard serves as the central centre for video management (Upload/Delete). The core functionality is located in the VideoClips Page which is based on the combination of the Search Bar, Parameter Controls (window size, re-order toggle) and the Video Player for streaming the results. The History Page offers the list view of past interactions. This modular interface design is to ensure a consistent user experience and navigating. The Interface Design is as shown in Figure 4.8.

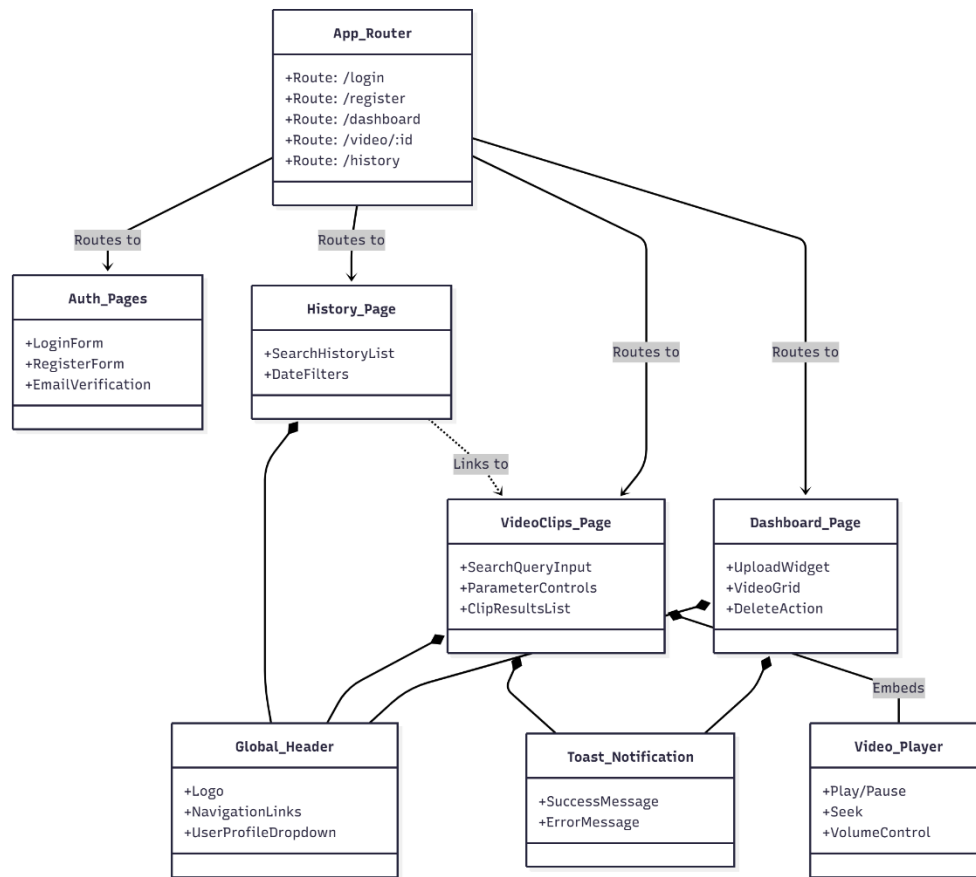


Figure 4.8: Interface Design Diagram

#### 4.9.2 Component Level design

Component level design is used to further refine the high level architectural view into the software modules and classes. It encapsulates the internal details of each subsystem and specifies the data structure, algorithms and interface signatures that are needed for implementation of the subsystem. This degree of design guarantees that the system is modular, maintainable and testable via decoupling distinct functionalities.

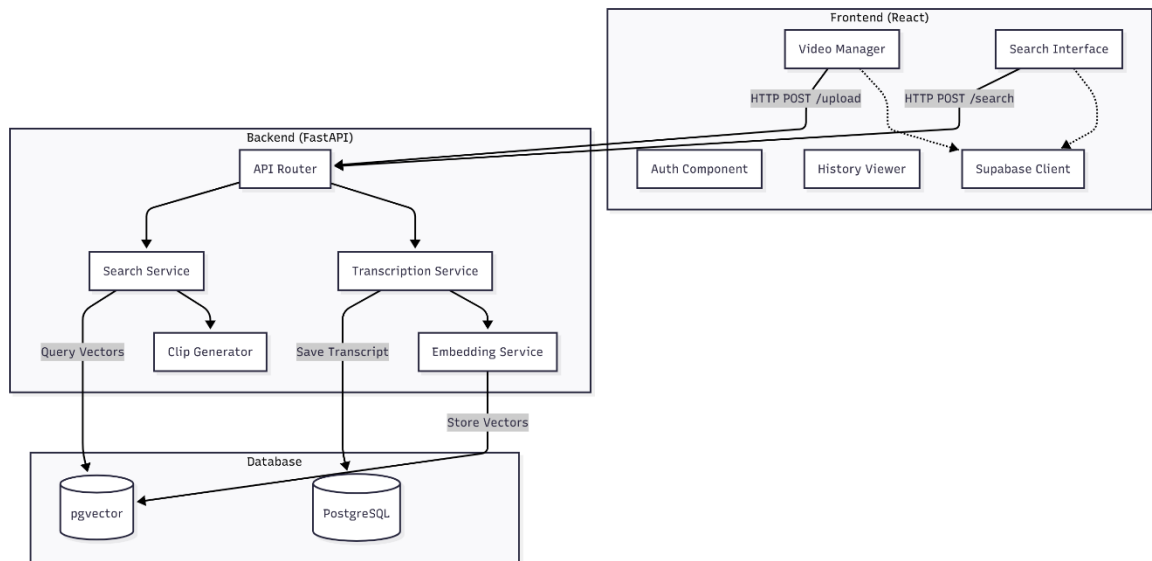


Figure 4.9: Component Level Design

## REFERENCES

- [1] Deepali Vora, Payal Kadam, Dadaso D Mohite, Nilesh Kumar, Nimit Kumar, Pratheeik Radhakrishnan & Shalmali Bhagwat.  
AI-driven video summarization for optimizing content retrieval and management through deep learning techniques. 2025,
- [2] Apostolidis, Evlampios; Adamantidou, Eleni; Metsai, Alexandros I.; Mezaris, Vasileios; Patras, Ioannis. Video Summarization Using Deep Neural Networks: A Survey, 1838 – 1863, 2021
- [3] Parul Saini, Krishan Kumar, Shamal Kashid, Ashray Saini & Alok Negi  
Video summarization using deep learning techniques: a detailed analysis and investigation, Pages: 12347–12385, 2023,
- [4] Payal Kadam; Deepali Vora; Sashikala Mishra; Shruti Patil; Ketan Kotecha; Ajith Abraham Recent Challenges and Opportunities in Video Summarization With Machine Learning Algorithms, Pages: 122762 – 122785, Vol. 10, 2022,
- [5] Roman Plaud, Jose-Luis Lisani, Two Deep Learning Solutions for Automatic Blurring of Faces in Videos, 2024,
- [6] Andisani Nemavhola, Serestina Viriri, Colin Chibaya, A Scoping Review of Literature on Deep Learning Techniques for Face Recognition, 2025
- [7] G.J. Sullivan; T. Wiegand, Video Compression - From Concepts to the H.264/AVC Standard, Pages: 18-31, 2005
- [8] "CapCut - Video Editor", Bytedance pte. Ltd., 2024. [Online]. Available: <https://play.google.com/store/apps/details?id=com.lemon.lvoverseas&hl=en&gl=US>.
- [9] “Notta - Dictée & Transcription”, Notta pte. Ltd., 2025. [Online]. Available: <https://play.google.com/store/apps/details?id=com.langogo.transcribe&hl=fr>
- [10] “Summary: AI Video Summary”, GODHITECH JSC, 2025, [Online]. Available: <https://play.google.com/store/apps/details?id=com.godhitech.summarize.quiz.mindmap&hl=fr>
- [11] “Video Editor & Maker – InShot”, InShot Video Editor, 2014. [Online]. Availabe: <https://play.google.com/store/apps/details?id=com.camerasideas.instashot&hl=en>